



Processi

Gestire l'esecuzione

Processi

- Un processo costituisce l'unità base di esecuzione di un applicativo nel contesto di un sistema operativo
 - Esso viene identificato in modo univoco a livello di Sistema Operativo tramite un numero intero PID – Process ID
 - Definisce uno spazio di indirizzamento all'interno del quale possono operare uno o più thread, flussi di esecuzione schedulabili indipendentemente
 - Lo spazio di indirizzamento fornisce un meccanismo naturale di separazione (**isolamento**)
 - Allo scopo di evitare che le attività nel contesto di un processo possano "disturbare" quelle di altri processi

Processi e isolamento

- Il livello di isolamento offerto dal concetto di processo è parziale
 - Due processi possono interferire attraverso il file system, sia a livello di file "dati" che a livello di eseguibili e librerie dinamiche
 - Il sistema di autenticazione/autorizzazione/accounting è una seconda fonte di possibile interferenza
 - Il sottosistema di rete è un'altra fonte di potenziale interferenza
 - In generale, l'accesso alle periferiche di sistema ed alle risorse centralizzate può causare incompatibilità
- In alcune situazioni, il progettista vuole esplicitamente ridurre il livello di isolamento tra due o più processi
 - Offrendo un meccanismo controllato di comunicazione in grado di superare i limiti imposti dalla separazione degli spazi di indirizzamento
- I sistemi operativi offrono, a questo proposito, meccanismi opportuni che ricadono sotto il nome generico di IPC
 - Inter-Process Communication

Concorrenza e processi

- L'uso dei thread permette di sfruttare le risorse computazionali presenti in un elaboratore
 - La presenza di uno spazio di indirizzamento condiviso facilita la coordinazione e la comunicazione
- Ci sono situazioni in cui la presenza di un singolo spazio di indirizzamento non è possibile o desiderabile
 - Riutilizzo di programmi esistenti
 - Scalabilità su più computer
 - Sicurezza

Concorrenza e processi

- È possibile decomporre un sistema complesso in un insieme di processi collegati
 - Creandoli a partire da un processo genitore
 - Permettendo la cooperazione indipendentemente dalla loro genesi
- Ad ogni processo è associato almeno un thread (primary thread)
 - Un sistema multiprocesso è intrinsecamente concorrente
 - Solleva gli stessi problemi di interferenza e necessità di coordinamento

Processi in Windows

- Costituiscono entità separate, senza relazioni di dipendenza esplicita tra loro
- La funzione **CreateProcess(...)**
 - Crea un nuovo spazio di indirizzamento vuoto
 - Lo inizializza con l'immagine di un eseguibile
 - Crea il thread primario al suo interno
 - Avvia il thread primario (attraverso la chiamata di `_crtstartup`, che prepara l'ambiente e chiama `main()`)
 - Quando `main()` termina, `_crtstartup` si occupa di chiamare `exit()` per pulire e restituire il codice di uscita al Sistema Operativo
- Il processo parte "pulito", ossia è l'immagine esatta dell'eseguibile caricato
- Il processo figlio può condividere variabili d'ambiente ed handle a file, semafori, pipe ...
 - ... ma non può condividere handle a thread, processi, librerie dinamiche e regioni di memoria

Creare processi in Windows

```
#include <windows.h>
#include <stdio.h>
#include <tchar.h>

void _tmain( int argc, TCHAR *argv[] )
{
    STARTUPINFO si;
    PROCESS_INFORMATION pi;
    ZeroMemory( &si, sizeof(si) );
    si.cb = sizeof(si);
    ZeroMemory( &pi, sizeof(pi) );
    if( argc != 2 ){
        printf("Usage: %s [cmdline]\n", argv[0]);
        return;
    }
    //continua...
```

Creare processi in Windows

```
// Start the child process
if( !CreateProcess(
    NULL,      // No module name (use command line)
    argv[1],  // Command line
    NULL,      // Process handle not inheritable
    NULL,      // Thread handle not inheritable
    FALSE,     // Set handle inheritance to FALSE
    0,         // No creation flags
    NULL,      // Use parent's environment block
    NULL,      // Use parent's starting directory
    &si,        // Pointer to STARTUPINFO struct
    &pi )       // Pointer to PROCESS_INFORMATION struct
) {
    printf( "CreateProcess failed (%d).\n", GetLastError() );
    return;
}
//...continua...
```

Creare processi in Windows

```
// Wait until child process exits.  
WaitForSingleObject( pi.hProcess, INFINITE );  
  
// Close process and thread handles.  
CloseHandle( pi.hProcess );  
CloseHandle( pi.hThread );  
}
```

```
typedef struct _PROCESS_INFORMATION {  
    HANDLE hProcess;  
    HANDLE hThread;  
    DWORD  dwProcessId;  
    DWORD  dwThreadId;  
} PROCESS_INFORMATION, *PPROCESS_INFORMATION, *LPPROCESS_INFORMATION;
```

Processi in Linux

- Si crea un processo figlio con la system call **fork()**
 - Crea un nuovo spazio di indirizzamento «identico» a quello del processo genitore: duplicazione dello spazio di indirizzamento, 2 copie identiche e indipendenti
 - I due processi condividono i riferimenti alle stesse pagine di memoria fisica
 - Il figlio inizia la propria computazione, con un singolo thread, trovandosi già uno stack popolato con la storia delle chiamate effettuate nel padre, uno heap con della memoria allocata, codice e spazio globale nello stesso stato in cui erano nel padre
- Dopo l'esecuzione di **fork()**, tutte le pagine sono marcate con il flag **CopyOnWrite**
 - Eventuali scritture comportano, l'aggiornamento del flag, la duplicazione della pagina e la separazione tra i due spazi di indirizzamento
 - Diversamente da Windows, un processo non parte "pulito", ma parte con una storia.

Nessun
parametro

Uscendo da `fork()` si
potrebbe fare `return`
e tornerai alla
procedura
chiamante...

fork()

Potrebbe restituire solo 1 volta -1 se il Sistema Operativo non è in grado di creare un nuovo processo

- Viene chiamata 1 volta e ritorna 2 volte:
 - Nello spazio di indirizzamento del processo padre ritorna l'identificatore (PID) del processo figlio creato
 - Nello spazio di indirizzamento del processo figlio ritorna 0. Il figlio per sapere il suo PID deve chiamare la system call getpid() e per sapere il PID del suo genitore deve chiamare la system call getppid().
- Questo determina una forte relazione tra padre e figlio.

Creazione di Processi

- Le funzioni **exec*()** sostituiscono l'attuale immagine di memoria dello spazio di indirizzamento
 - Ri-inizializzandola a quella descritta dall'eseguibile indicato come parametro
 - Differiscono tra loro nel modo di gestire i parametri ricevuti

Esempio

```
int main ( const int argc, const char* const argv[] ) {
    pid_t  childPid = fork();
    switch (childPid) {
        case -1:
            puts( "parent: error: fork failed!" );break;
        case  0:
            puts( "child: here (before execl)!" );
            if (execl( "./ch.exe", "./ch.exe", 0 )==-1)
                perror( "child: execl failed:" );
            puts( "child: here (after execl)!" );
            //non si dovrebbe arrivare qui
            break;
        default:
            printf( "par: child pid=%d \n", ret );
            break;
    }
    return 0;
}
```

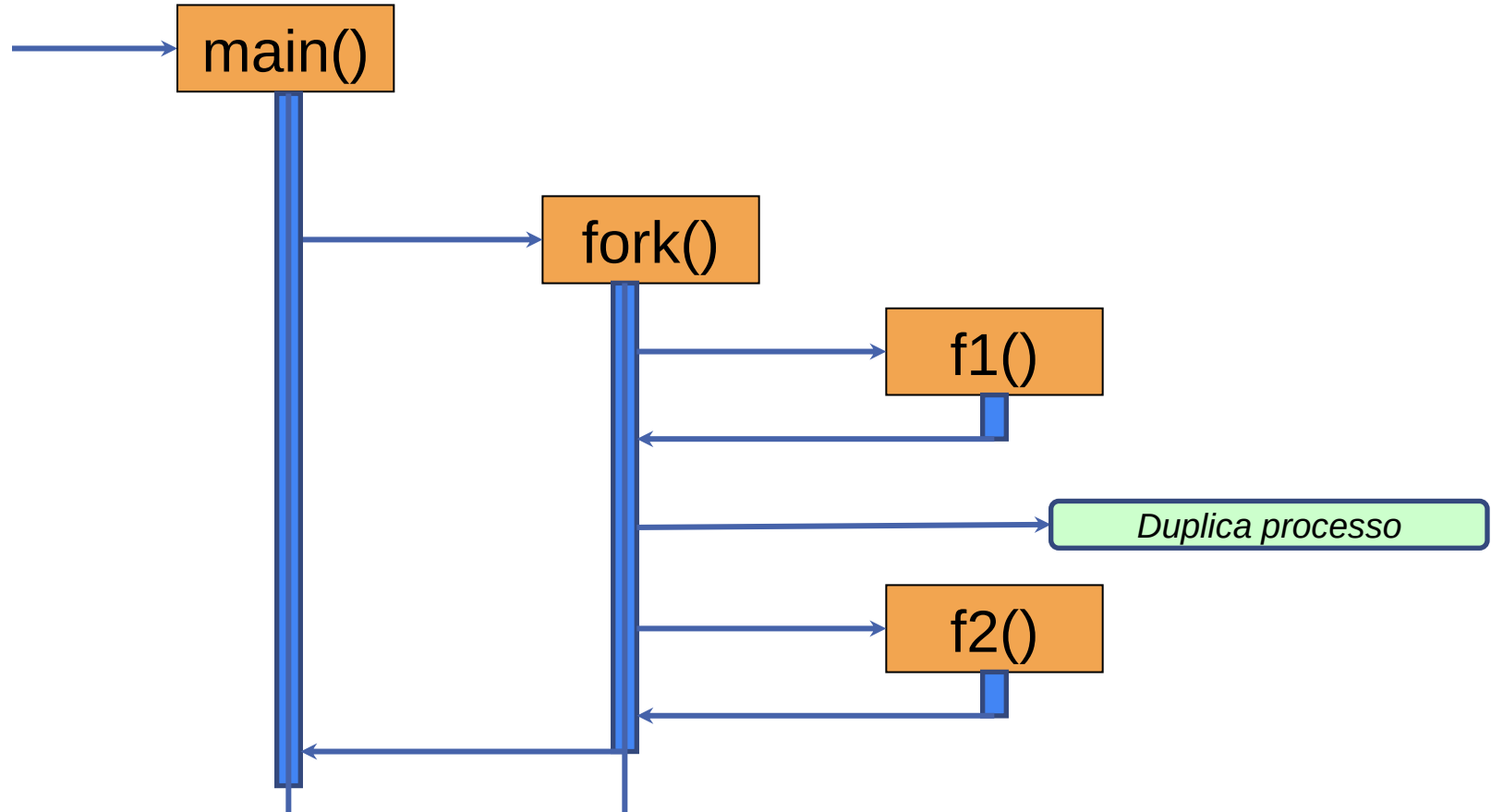
Fork() e thread

- Nel caso di programmi concorrenti, l'esecuzione di fork() crea un problema
 - Il processo figlio conterrà un solo thread
 - Gli oggetti di sincronizzazione presenti nel padre possono trovarsi in stati incongruenti (ossia, che cosa succederebbe se un mutex era posseduto da un altro thread diverso da quello che ha fatto la fork?
Risposta: Deadlock)
- **int pthread_atfork(
void (*prepare)(void),
void (*parent)(void),
void (*child)(void)
);**
 - Registra un gruppo di funzioni che saranno chiamate in corrispondenza delle invocazioni a fork(), prima dell'invocazione della fork(), prepare(), solo dal padre al ritorno dalla fork(), parent(), e solo dal figlio al ritorno dalla fork(), child().
 - prepare() acquisisce tutti i mutex in modo che nessun altro thread possa modificarne lo stato prima della duplicazione
 - Parent() rilascia i mutex che sono stati acquisiti dalla funzione prepare()
 - Child() fa la stessa cosa, ma può anche essere utilizzato per resettare stati interni o risorse che non sono valide nel contesto del processo figlio.

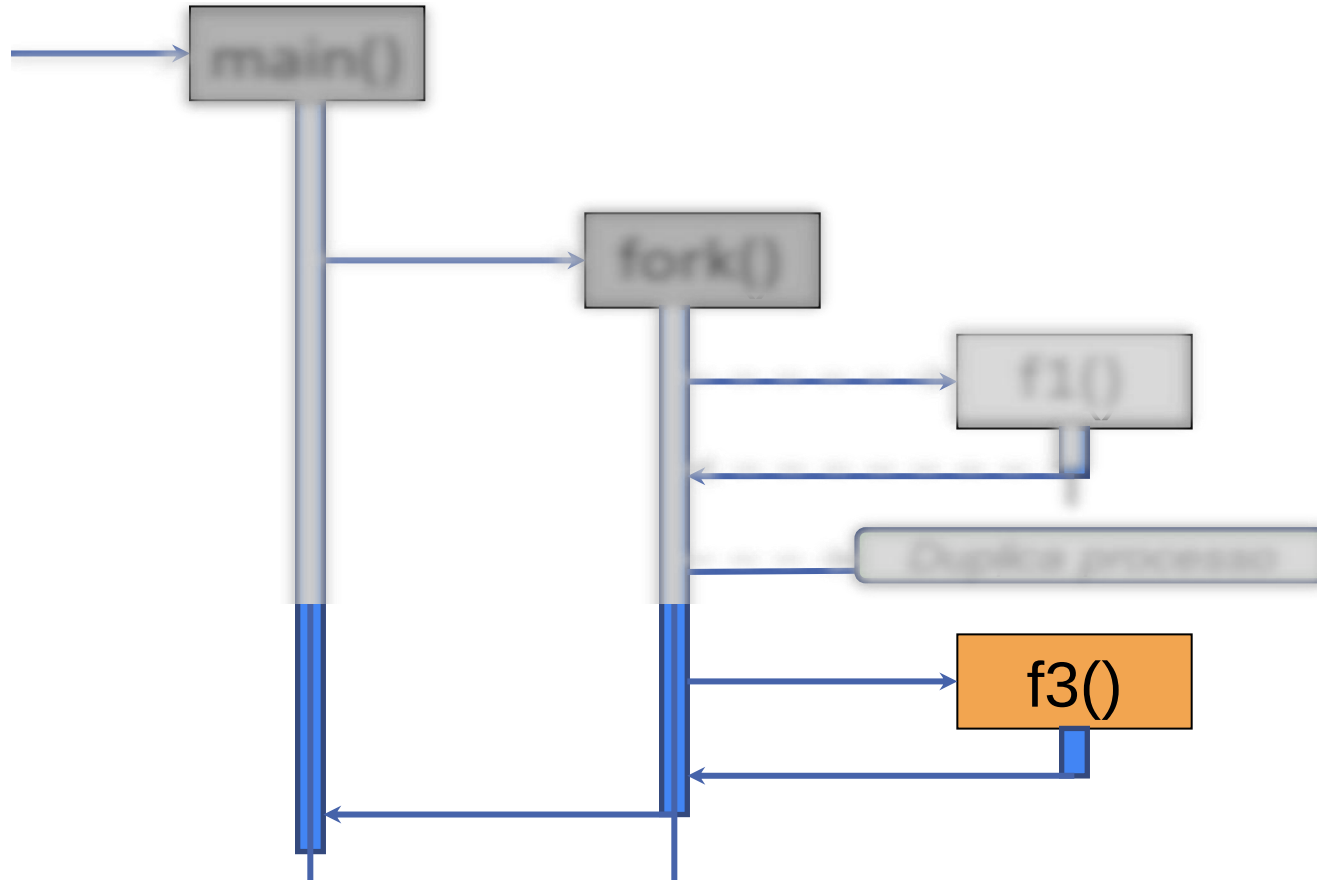
Esempio

```
void f1() { ... }  
void f2() { ... }  
void f3() { ... }  
  
int main() {  
    pthread_atfork(f1,f2,f3);  
    //...  
    int res= fork();  
    if (res== -1 ) { /* errore */ }  
    else if (res == 0) { /* child */ }  
    else {          /* parent */ }  
}
```

Processo genitore



Processo figlio



Terminare un processo

- Un processo continua la propria esecuzione fino a che non termina di propria volontà o viene terminato dall'esterno
 - Una opportuna system call permette di richiedere la terminazione del processo corrente e la conseguente deallocazione di tutte le risorse in uso (memoria, descrittori di file, lock e altri oggetti di sincronizzazione, primitive di IPC, socket di rete, ...) e la loro restituzione al sistema operativo, che potrà renderle disponibili ad altri processi

Terminare un processo

- Qualunque thread può chiedere la terminazione del processo
 - In Windows si invoca `ExitProcess(int status)`
 - In Linux la funzione `_exit(int status)`
- Entrambe causano l'IMMEDIATA terminazione di tutti gli altri thread associati al processo
 - Senza ulteriori possibilità di esecuzione
- Tutti gli handle (file, mutex, thread) aperti sono chiusi
- Tutte le risorse di memoria allocate vengono rilasciate
 - Nel caso di Windows, nel contesto del thread che ha eseguito la funzione `ExitProcess(...)` vengono rilasciate le DLL eventualmente caricate

Numero su 8 bit che convenzionalmente vale 0 se il processo è terminato con successo

Terminare un processo

- La funzione `_exit(int status)` ha lo scopo di terminare immediatamente il processo chiamante senza eseguire la pulizia standard del C Runtime (CRT). In particolare non vengono chiamati i distruttori di oggetti globali o statici
- Le librerie standard del C e del C++ offrono una soluzione portabile e più articolata
 - la funzione `void exit(int status)` definita in `<stdlib.h>`
 - la funzione `void std::exit(int status)` definita in `<cstdlib>`
- Queste supportano la possibilità di registrare una o più callback da invocare all'atto della terminazione
 - Tramite la funzione `int std::atexit(void (*callback)())`
 - Il valore restituito indica se la registrazione è andata a buon fine o meno

Gestire la terminazione

```
#include <iostream>
#include <cstdlib>

void atexit_handler_1() {
    std::cout << "at exit #1\n";
}

void atexit_handler_2() {
    std::cout << "at exit #2\n";
}

int main() {
    const int result_1 = std::atexit(atexit_handler_1);
    const int result_2 = std::atexit(atexit_handler_2);
    if ((result_1 != 0) || (result_2 != 0)) {
        std::cerr << "Registration failed\n";
        return EXIT_FAILURE;
    }
    std::cout << "returning from main\n";
    return EXIT_SUCCESS;
}
```

Gestire la terminazione

- Un programma termina anche quando la funzione principale ritorna
 - Questo è conseguenza del codice di startup inserito dalla libreria di supporto del linguaggio
 - Questa, inizializza l'ambiente di esecuzione, invoca la funzione `main(...)` e utilizza il valore da essa ritornato per invocare `exit(status)`
- Se si verifica un'eccezione non gestita nel thread principale o in uno secondario
 - Viene invocata la funzione `ExitProcess(...)` o `_exit(...)` con un codice di stato definito dalla libreria di esecuzione

Codice di ritorno

- Il valore restituito dalla funzione `exit(...)` è arbitrario
 - Vale una convenzione generale per la quale 0 indica una terminazione "pulita" e qualunque altro valore indica una terminazione con errore
 - Il significato di tale codice, tuttavia, non è oggetto di specifica da parte del sistema operativo
 - Occorre documentare opportunamente il valore e attenersi alle buone pratiche definiti in ciascun ambiente (OS + compilatore + libreria standard)

Processi in Rust

- Per la gestione dei processi, la standard library di Rust mette a disposizione il modulo **std::process**
 - I metodi offerti da Rust utilizzano al loro interno le system call esposte dal kernel del sistema operativo per gestire i processi in modo totalmente trasparente al Sistema Operativo
- La struct **Command** permette la creazione di un nuovo processo
 - Utilizza il pattern **builder** per **configurare**, **creare** ed **interagire** con il processo figlio
 - I metodi **arg()** ed **args()** possono essere utilizzati per passare al processo figlio rispettivamente uno o più argomenti
 - Il metodo **output()** genera il processo ed attende la sua terminazione ritornando un valore di tipo **Result<Output>**

```
pub struct Output {  
    pub status: ExitStatus,  
    pub stdout: Vec<u8>,  
    pub stderr: Vec<u8>,  
}
```

Processi in Rust

- Per la gestione dei processi, la standard library di Rust mette a disposizione il modulo `std::process`
 - I metodi offerti da rust utilizzano al loro interno le system call esposte dal kernel del sistema operativo per gestire i processi in modo totalmente trasparente al Sistema Operativo
- La struct **Command** permette la creazione di un nuovo processo
 - Utilizza il pattern builder per configurare, creare ed interagire con il processo figlio
 - I metodi `arg()` ed `args()` possono essere utilizzati per passare al processo figlio rispettivamente uno o più argomenti
 - Il metodo `output()` genera un processo che genera un output di tipo `Result<Output>`

```
pub struct Output {  
    pub status: ExitStatus,  
    pub stdout: Vec<u8>,  
    pub stderr: Vec<u8>,  
}
```

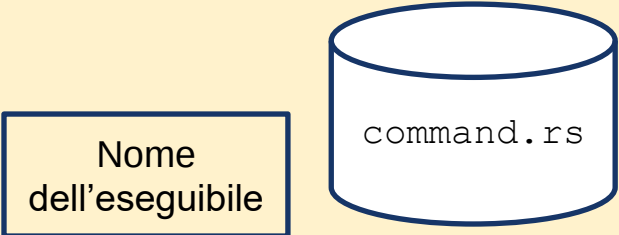
ExitStatus è una struttura che permette di avere informazioni sul codice di uscita del processo e, nel caso di sistemi Unix, sulla motivazione della sua terminazione (fine dell'esecuzione, terminazione forzata tramite un segnale, sospensione/continuazione a seguito di un segnale, eventuale presenza di una copia dello spazio di indirizzamento - *core dump*)

Processi in Rust

```
use std::process::Command;

fn main() {
    let output = if cfg!(target_os = "windows") {
        Command::new("cmd")
            .args(["/C", "echo hello"])
            .output()
            .expect("failed to execute process")
    } else {
        Command::new("sh")
            .arg("-c")
            .arg("echo hello")
            .output()
            .expect("failed to execute process")
    };

    println!("{:?}", output)
    // in Linux Output { status: ExitStatus(unix_wait_status(0)), stdout: "hello\n", stderr: "" }
    // in Windows Output { status: ExitStatus(ExitStatus(0)), stdout: "hello\r\n", stderr: "" }
}
```



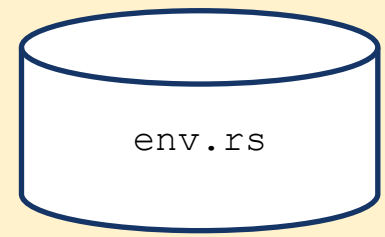
The diagram illustrates the mapping of the command name to the executable file. A rectangular box labeled "Nome dell'eseguibile" (Executable Name) is connected by a horizontal line to a cylindrical database icon labeled "command.rs". This represents the lookup of the command name "cmd" in the "command.rs" database to find the path to the executable.

Processi in Rust

- E' possibile configurare le variabili d'ambiente del processo prima di avviarlo
 - Per default, esso eredita quelle del processo corrente
 - I metodi `env<K,V>(&mut self, key: K, val: V)` e `envs<I,K,V>(&mut self, vars: I)` permettono di effettuare aggiunte o modifiche
 - I metodi `env_remove<K>(&mut self, key: K)` e `env_clear(&mut self)` permettono di eliminare una/tutte le variabili d'ambiente
 - E' possibile conoscere l'elenco delle variabili d'ambiente usate da una struct di tipo `Command` con il metodo `get_envs(&self)` che restituisce un iteratore a tuple formate dalle coppie chiave/valore
- E' possibile **ridirigere** i flussi standard di ingresso/uscita, tramite i metodi `stdin(...)`, `stdout(...)` e `stderr(...)`
 - E' possibile passare loro una delle seguenti opzioni:
 - `inherit()` → Il processo figlio eredita il descrittore in uso nel processo genitore
 - `piped()` → Verrà creata una pipe monodirezionale, un'estremità della quale sarà passata al processo figlio mentre l'altra sarà memorizzata nella struttura restituita dal comando di avvio
 - `null()` → Il flusso sarà ignorato
 - default ad `inherit` quando si usa `spawn` o `status`, e default a `piped` quando si usa `output`.

```
use std::process::Stdio;  
use std::process::Command;
```

```
fn main() {  
  
    let _output = Command::new("ls")  
        .env("PATH", "/bin")  
        .stdout(Stdio::inherit())  
        .output()  
        .expect("ls command failed to start");  
}
```



```

use std::process::Command;
use std::env; // Per leggere le variabili d'ambiente del processo corrente, se necessario

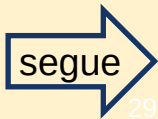
fn main() {
    // Esempio 1: Avviare un comando semplice (ad esempio 'echo' o 'printenv')
    // e passargli una variabile d'ambiente specifica.
    // useremo 'printenv' su Linux per vedere l'ambiente.

    let mut command = Command::new("sh");
    command.arg("-c").arg("echo 'Hello from child process!' && printenv MY_CUSTOM_VAR OTHER_VAR");

    let output = command
        // Aggiungiamo una variabile d'ambiente specifica per il processo figlio
        .env("MY_CUSTOM_VAR", "Il mio valore personalizzato")
        // Possiamo anche aggiungere più variabili
        .env("OTHER_VAR", "Un altro valore")
        // Oppure possiamo cancellare una variabile d'ambiente esistente dal processo padre
        .env_remove("PATH") // Rimuove 'PATH' dall'ambiente del figlio, se esiste nel padre
        .output() // Esegue il comando e cattura l'output
        .expect("Failed to execute command");

    // Stampiamo l'output standard e l'errore standard del processo figlio
    println!("Standard Output:\n{}", String::from_utf8_lossy(&output.stdout));
    println!("Standard Error:\n{}", String::from_utf8_lossy(&output.stderr));
    println!("Stato di uscita: {}", output.status);
}

```



```
// Si potrebbe volere un ambiente "pulito" e aggiungere solo le variabili necessarie.  
// Questo è utile per isolare il processo figlio.
```

```
let mut command_clean = Command::new("sh");  
command_clean.arg("-c").arg("echo 'Clean environment test' && printenv CUSTOM_ONLY_VAR");
```

```
let output_clean = command_clean  
    .env_clear() // Cancella TUTTE le variabili d'ambiente ereditate dal processo padre  
    .env("CUSTOM_ONLY_VAR", "Solo questa variabile") // Aggiunge solo quelle che vuoi  
    .output()  
    .expect("Failed to execute clean command");
```

```
println!("Standard Output (ambiente pulito):\n{}", String::from_utf8_lossy(&output_clean.stdout));  
println!("Standard Error (ambiente pulito):\n{}", String::from_utf8_lossy(&output_clean.stderr));  
println!("Stato di uscita (ambiente pulito): {}", output_clean.status);
```

```
println!("\n--- Variabili d'ambiente del processo padre (questo programma) ---");  
// Si possono vedere le variabili del processo padre (questo programma) usando std::env::vars()  
// Non esiste 'MY_CUSTOM_VAR', perché è stata passata solo al figlio.  
for (key, value) in env::vars() {  
    //if key == "PATH" || key == "HOME" || key == "USER"  
    {  
        println!("{}", key, value);  
    }  
}  
}
```



```
use std::process::Stdio;
use std::process::Command;
```

```
Output { status: ExitStatus(unix_wait_status(0)), stdout: "", stderr: "" }
Hello, world!
Output { status: ExitStatus(unix_wait_status(0)), stdout: "", stderr: "" }
Output { status: ExitStatus(unix_wait_status(0)), stdout: "Hello, world!\n", stderr:
```

```
fn main() {
```

```
    let mut output = Command::new("echo")
```

```
        .arg("Hello, world!")
```

```
        .stdout(Stdio::null()) // This stream will be ignored.
```

```
                                // equivalent of attaching the stream to /dev/null
```

```
        .output()
```

```
        .expect("Failed to execute command");
```

```
    println!("{:?}", output);
```

```
    output = Command::new("echo")
```

```
        .arg("Hello, world!")
```

```
        .stdout(Stdio::inherit()) // The child inherits from the corresponding parent descriptor
```

```
        .output()
```

```
        .expect("Failed to execute command");
```

```
    println!("{:?}", output);
```

```
    output = Command::new("echo")
```

```
        .arg("Hello, world!")
```

```
        .stdout(Stdio::piped()) // A new pipe should be arranged to connect the parent and child processes
```

```
        .output()
```

```
        .expect("Failed to execute command");
```

```
    println!("{:?}", output);
```

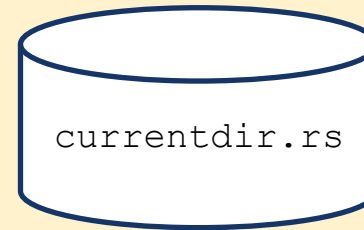
```
}
```

stdout.rs

- E' possibile modificare la cartella in cui si avvia il processo figlio tramite il metodo `current_dir<P: AsRef<Path>>(&mut self, dir: P)`

```
use std::process::Stdio;
use std::process::Command;

fn main() {
    let _output = Command::new("ls")
        .current_dir("/bin")
        .stdout(Stdio::inherit())
        .output()
        .expect("ls command failed to start");
}
```



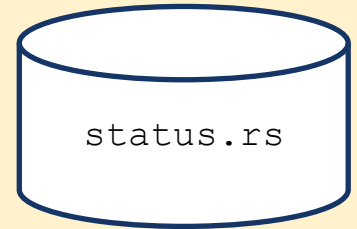
status()

- Il metodo **status(&mut self)** **avvia** il processo e ne **attende** la **terminazione**
 - Esso restituisce un valore di tipo **Result<ExitStatus>**, dove **ExitStatus** è una struttura che permette di avere informazioni sul codice di uscita del processo e, nel caso di sistemi Unix, sulla motivazione della sua terminazione (fine dell'esecuzione, terminazione forzata tramite un segnale, sospensione/continuazione a seguito di un segnale, eventuale presenza di una copia dello spazio di indirizzamento - *core dump*)

```
use std::process::Command;

fn main() {
    // Esegui il comando ls e ottieni il suo stato di uscita
    let child = Command::new("ls")
        .status()
        .expect("failed to execute process");

    // Stampa lo stato di uscita del processo
    if child.success() {
        println!("Process exited successfully with status: {}", child);
    } else {
        println!("Process exited with failure status: {}", child);
    }
}
```



spawn()

- Il metodo `spawn(&mut self)` avvia invece il processo senza attenderne la terminazione
 - Esso restituisce un valore di tipo `Result<Child>`, dove `Child` è una struttura che consente di rappresentare ed interagire con il processo figlio

Le 3 modalità



- **output:** genera il processo figlio ed attende la sua terminazione ritornando un valore di tipo **Result<Output>**; **stdin** e **stdout** di default a **piped()**.
- **status:** genera il processo figlio ed attende la sua terminazione ritornando un valore di tipo **Result<ExitStatus>**; **stdin** e **stdout** di default a **inherit()**.
- **spawn:** genera il processo figlio e restituisce un valore di tipo **Result<Child>** e non attende la terminazione del processo; **stdin** e **stdout** di default a **inherit()**.

Processi in Rust

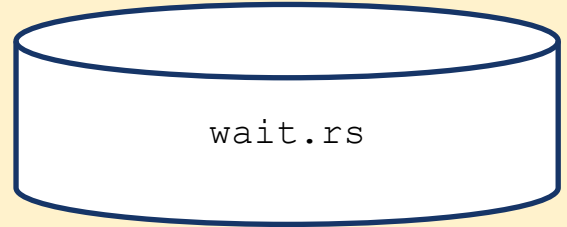
- La struttura `Child` offre vari meccanismi per controllare e condizionare lo svolgimento del processo figlio
 - Attraverso i campi `stdin`, `stdout`, `stderr` è possibile fare accesso alle handle dei relativi flussi, qualora siano stati catturati
 - Il metodo `id(&self)` restituisce l'identificativo univoco assegnato dal sistema operativo
 - Il metodo `wait(&mut self)` ne attende la terminazione, restituendo il codice di uscita
 - Il metodo `wait_with_output(&mut self)` chiude il flusso di ingresso del processo figlio, ne attende la terminazione e raccoglie quanto non ancora letto dei flussi di uscita ed errore in una struttura di tipo `Output`
 - il metodo `kill()` ne forza la terminazione

```

use std::process::Command;

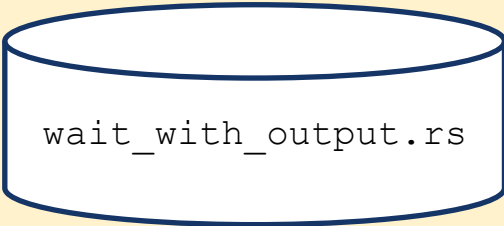
fn main() {
    // Crea un'istanza di Command e gestisce il risultato di spawn
    match Command::new("ppp")
        .arg("Hello World")
        .spawn() {
        Ok(mut child) => {
            // Il processo è stato avviato con successo
            match child.wait() {
                Ok(status) => {
                    if status.success() {
                        println!("Il processo è terminato con successo");
                    } else {
                        println!("Il processo è terminato con errore: {:?}", status.code());
                    }
                }
                Err(e) => println!("Errore nell'attesa del processo: {}", e)
            }
        }
        Err(e) => {
            // Gestione dell'errore di spawn
            println!("Impossibile avviare il processo: {}", e);
            // Qui puoi vedere il tipo specifico di errore
            if e.kind() == std::io::ErrorKind::NotFound {
                println!("Il comando specificato non esiste!");
            }
        }
    }
}

```



```
use std::process::{Command, Stdio};
```

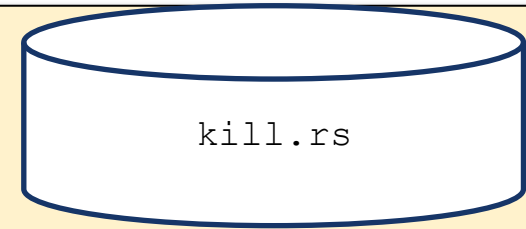
```
fn main() {  
    let child_process = Command::new("ls")  
        .arg("-l")  
        .arg("-a")  
        .stdout(Stdio::piped())  
        .spawn()  
        .expect("Impossibile avviare il processo 'ls'");  
  
    let output = child_process.wait_with_output().expect("Impossibile ottenere l'output del processo");  
  
    if output.status.success() {  
        let stdout = String::from_utf8_lossy(&output.stdout);  
        println!("Output del processo 'ls':\n{}", stdout);  
    } else {  
        let stderr = String::from_utf8_lossy(&output.stderr);  
        println!("Errore durante l'esecuzione del processo 'ls': {}", stderr);  
    }  
}
```



wait_with_output.rs

```
use std::process::{Command, Stdio};
```

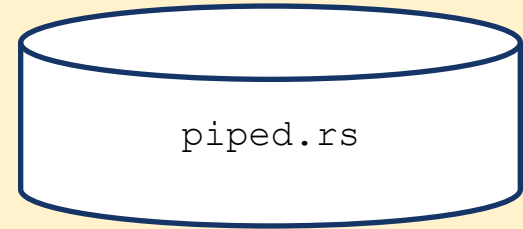
```
fn main() {  
    // Esegue un comando esterno  
    let mut child = Command::new("sleep")  
        .arg("10")  
        .stdout(Stdio::piped())  
        .spawn()  
        .expect("Failed to start command");  
  
    // Ottiene l'ID del processo  
    let pid = child.id();  
  
    // Termina il processo  
    match child.kill() {  
        Ok(_) => println!("Process {} killed", pid),  
        Err(err) => println!("Error killing process {}: {}", pid, err),  
    }  
  
    // Attendere che il processo figlio termini  
    let exit_status = child.wait();  
    match exit_status {  
        Ok(status) => println!("Exit status: {}", status),  
        Err(err) => println!("Error waiting for child: {}", err),  
    }  
}
```



Process 1989 killed
Exit status: signal: 9 (SIGKILL)

Ridiregere i flussi di ingresso/uscita

```
use std::io::prelude::*;
use std::process::{Command, Stdio};
fn main() {
    let process = match Command::new("rev")
        .stdin(Stdio::piped())
        .stdout(Stdio::piped())
        .spawn()
    {
        Err(err) => panic!("couldn't spawn rev: {}", err),
        Ok(process) => process,
    };
    match process.stdin.unwrap().write_all("'isoc ovitrevid im non ehc innaaaa
onarE".as_bytes()) {
        Err(why) => panic!("couldn't write to stdin: {}", why),
        Ok(_) => println!("sent text to rev command"),
    }
    let mut child_output = String::new();
    match process.stdout.unwrap().read_to_string(&mut child_output) {
        Err(err) => panic!("couldn't read stdout: {}", err),
        Ok(_) => print!("Output from child process is:\n{}", child_output),
    }
}
```



```
sent text to rev command
Output from child process is:
Erano aaaanni che non mi divertivo cosi'
```

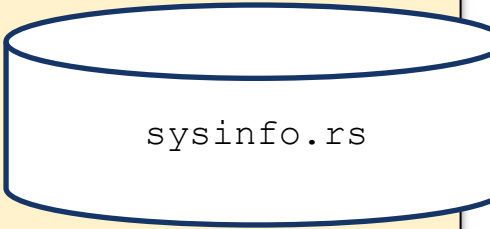
sysinfo

- Il crate sysinfo permette di accedere alle informazioni dei processi così come esposte dai diversi sistemi operativi

[dependencies]

sysinfo = "0.30.12"

```
use sysinfo::{Pid, System};
fn main() {
    let mut sys = System::new();
    // First we update all information of our `System` struct.
    sys.refresh_all();
    println!("=> system:");
    // RAM and swap information:
    println!("total memory: {} bytes", sys.total_memory());
    println!("used memory : {} bytes", sys.used_memory());
    println!("total swap   : {} bytes", sys.total_swap());
    println!("used swap    : {} bytes", sys.used_swap());
    // Display system information:
    println!("System name:      {:?}", System::name());
    println!("System kernel version:  {:?}", System::kernel_version());
    println!("System OS version:     {:?}", System::os_version());
    println!("System host name:      {:?}", System::host_name());
    // Number of CPUs:
    println!("Number CPUs: {}", sys.cpus().len());
    // Display processes ID, name na disk usage:
    for (pid, process) in sys.processes() {
        println!("[{pid}] {} {:?}", process.name(), process.disk_usage());
    }
}
```



sysinfo.rs

Interazioni tra `fork()`, `stdio` e `exit()`

- Se, in Linux, un processo esegue `fork()`, tutto lo stato della sua memoria sarà duplicato
 - Nel caso in cui tale processo avesse avuto, nei buffer di IO contenuto pendente, tale contenuto verrà scaricato sul dispositivo corrispondente sia dal processo padre che da quello figlio, non appena eseguiranno una operazione di `flush()` o satureranno tale buffer con ulteriori operazioni
- Il problema è più evidente quando l'output di un programma è rediretto verso un file
 - Questo abilita una modalità di scrittura a blocchi (al posto di quella standard a linee) che aumenta la probabilità di osservare tale fenomeno
 - Prima di eseguire `fork()`, può essere conveniente eseguire un `flush()` di tutti i file aperti (compresi `std::cout` e `std::cerr`)

Terminare un processo

- La funzione `std::process::exit(code: i32) -> !` termina immediatamente il processo corrente, con tutti i thread presenti al suo interno
 - Nessun distruttore presente sullo stack del thread corrente né degli altri thread viene eseguito
 - E' responsabilità del programmatore invocare questa funzione solo in punti in cui si abbia la ragionevole sicurezza che tutto ciò che doveva essere liberato sia stato liberato
 - Nonostante il valore ricevuto sia a 32 bit, nella maggior parte dei sistemi Unix-like, solo gli 8 bit meno significativi sono effettivamente passati al sistema operativo
- La funzione `std::process::abort() -> !` termina immediatamente il processo corrente, con tutti i thread presenti al suo interno
 - Non ci permette di mettere un codice di errore, ma viene messo uno di default
 - E forza un codice di errore che viene interpretato come interruzione anomala
 - Valgono le stesse cautele espresse per la funzione soprastante
- La macro `panic!(...)` causa la contrazione dello stack corrente, con l'esecuzione di tutti i distruttori posti al suo interno, senza determinare la terminazione del processo, a meno che la chiamata avvenga nel contesto del thread principale (diversamente dal C++ dove l'attivazione di un thread è incapsulata in un blocco try-catch che intercetta il fallimento del thread con la chiamata della funzione `terminate()` che causa la fine del processo).

Gestire altri processi

- Per processi creati da un programma Rust, le informazioni sono acquisibili acquisendo un oggetto Child ottenuto da una chiamata spawn().
- Per processi creati esternamente occorre considerare le system call offerte dai sistemi operativi per attendere la terminazione di un processo di cui si conosce la handle
 - Tale attesa non comporta consumo di CPU e termina quando il processo osservato termina per qualunque motivo
- Le API offerte dai diversi sistemi operativi differiscono alquanto
 - In Windows si utilizzano `WaitForSingleObject(...)` / `WaitForMultipleObjects(...)` e `GetExitCodeProcess(...)`
 - In Linux, si utilizzano `wait(...)`, `waitpid(...)` e `waitid(...)`

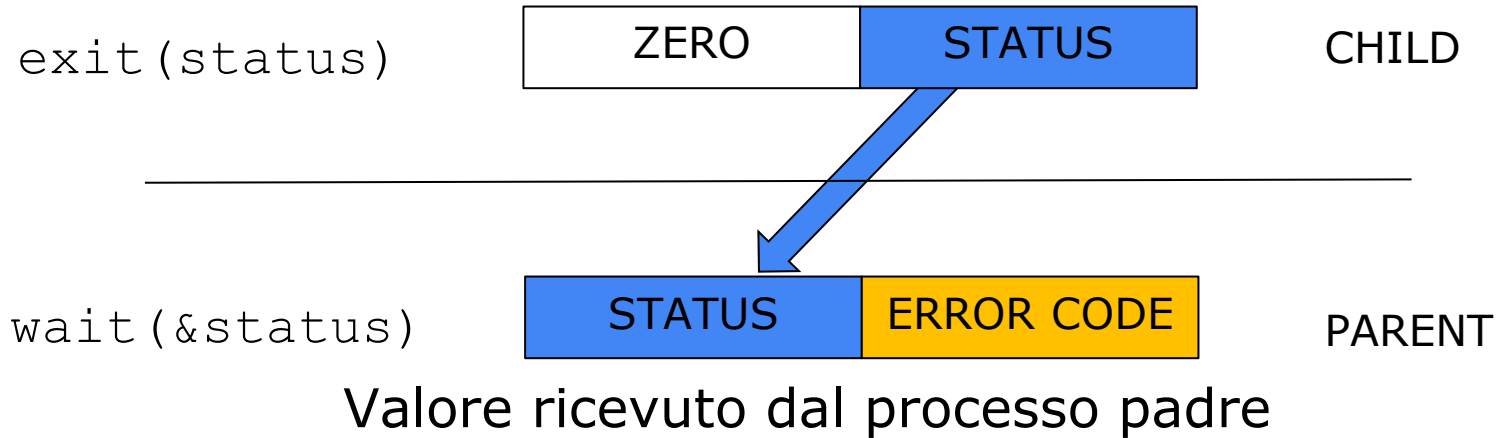
Gestire altri processi

- La funzione `pid_t wait(int *status)`, definita in `<sys/wait.h>`, guarda i propri figli e opera come segue:
 - Se un processo non ha processi figli, la funzione ritorna -1
 - Se nessuno dei figli dell'attuale processo è terminato, **la chiamata si blocca in attesa** di tale evento
 - In presenza di un figlio terminato:
 - Se status non è nullo, al suo interno viene indicata la motivazione che ne ha causato la terminazione
 - Il sistema aggiorna i contatori di utilizzo del sistema (CPU/memoria/IO/rete) del processo corrente aggiungendo quelli del processo figlio
 - Viene restituito il ProcessID del figlio terminato
 - Eventuali ulteriori chiamate a `wait(...)` da parte del processo corrente non forniranno più indicazioni relative a questo processo figlio

Gestire altri processi

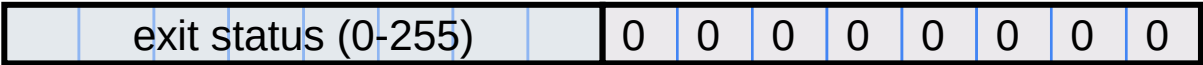
- La funzione `pid_t waitpid(pid_t pid, int *status, int options)` permette di indicare uno specifico processo figlio
 - E anche di eseguire una verifica non bloccante del suo stato attuale
 - Attenzione all'uso del polling che può causare consumi eccessivi di CPU e batteria!
- L'intero a cui punta `status` (se non è nullo) viene inizializzato con un valore a 16 bit
 - Il suo contenuto è definito in una serie di sotto-campi

Valore ritornato al processo padre



Stato terminale di un processo in Linux

Terminazione normale



Terminazione a seguito di un segnale

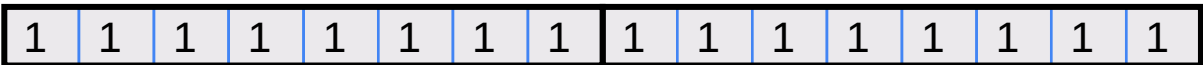


Core dumped flag

Bloccato da un segnale



Continuato da un segnale



Core Dump

- Corrisponde ad una copia dello stato della memoria di un processo al momento del suo fallimento. Utile per identificare la causa del fallimento.
- Il core dump viene generato quando un programma riceve un signal che indica un errore fatale (ad es. SIGSEGV per una segment violation, SIGABRT per un abort, SIGILL per una illegal instruction, SIGQUIT generato da tastiera con CTRL+\).
- In queste situazioni il kernel del sistema operativo può generare un file core dump.

```
use std::process::Command;
```

```
fn main() {  
    // Esegui un comando che si sa restituirà un valore di uscita specifico  
    let command = Command::new("ls")  
        .arg("/nonexistent_directory")  
        .spawn();  
  
    match command {  
        Ok(mut child) => {  
            // Attendere il completamento del processo figlio e ottenere l'ExitStatus  
            match child.wait() {  
                Ok(exit_status) => {  
                    // Verificare se il processo è terminato correttamente o con un errore  
                    if exit_status.success() {  
                        println!("Il processo è terminato correttamente.");  
                    } else {  
                        println!("Il processo è terminato con errore.");  
                    }  
  
                    // Ottenere il codice di uscita del processo se disponibile  
                    if let Some(code) = exit_status.code() {  
                        println!("Codice di uscita del processo: {}", code);  
                    } else {  
                        println!("Il processo è stato terminato da un segnale.");  
                    }  
                }  
                Err(e) => {  
                    eprintln!("Errore durante l'attesa del processo: {}", e);  
                }  
            }  
        }  
        Err(e) => {  
            eprintln!("Errore durante l'avvio del processo: {}", e);  
        }  
    }  
}
```



exitstatus.rs



```

use std::process::{Command, Stdio};
use std::os::unix::process::ExitStatusExt;
fn main() {
    let mut child_process = Command::new("timer")
        .env("PATH", "./target/debug/")
        .stdin(Stdio::piped())
        .stdout(Stdio::piped())
        .spawn()
        .unwrap();
    let child_pid = child_process.id(); // Ottieni l'ID del processo figlio
    let result = Command::new("kill") // Invia il segnale SIGTERM al processo figlio
        .arg("-15") // Codice del segnale SIGTERM
        .arg(child_pid.to_string())
        .status();
    match result {
        Ok(status) => {
            if status.success() {
                println!("Segnale SIGTERM inviato al processo figlio.");
            } else {
                println!("Errore nell'invio del segnale.");
            }
        }
        Err(_) => { println!("Errore nell'esecuzione del comando kill."); }
    }
    match child_process.wait() {
        Ok(exit_status) => {
            if exit_status.success() {
                println!("Il processo è terminato correttamente.");
            } else {
                println!("Il processo è terminato con errore: {:?}", exit_status);
            }
            if let Some(code) = exit_status.code() {
                println!("Codice di uscita del processo: {}", code);
            } else {
                if let Some(signal) = exit_status.signal() {
                    println!("Processo terminato dal segnale: {}", signal);
                }
                if exit_status.core_dumped() {
                    println!("Il processo ha generato un core dump");
                }
            }
        }
        Err(e) => { eprintln!("Errore durante l'attesa del processo: {}", e); }
    }
}

```

signal.rs

timer.rs

```

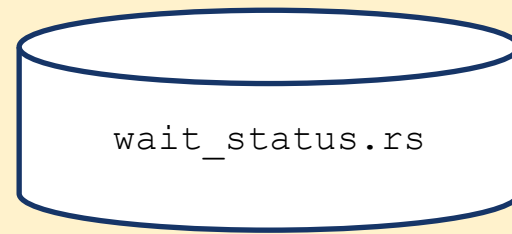
use std::time::Duration;
use std::thread::sleep;
fn main() {
    for i in 1..100 {
        sleep(Duration::from_secs(1));
        println!("{}", i);
    }
}

```

```

use std::process::{Command, Stdio};
use std::os::unix::process::ExitStatusExt;
fn main() {
    let mut child_process = Command::new("exit0")
        .env("PATH", "./target/debug/")
        .stdin(Stdio::piped())
        .stdout(Stdio::piped())
        .spawn()
        .unwrap();
    let _ = child_process.id(); // Ottieni l'ID del processo figlio
    match child_process.wait() {
        Ok(exit_status) => {
            println!("Exited with exit code: {}", exit_status);
            // Verificare se il processo è terminato correttamente o con un errore
            if exit_status.success() {
                println!("Il processo è terminato correttamente.{:?}",exit_status);
            } else {
                println!("Il processo è terminato con errore: {:?}",exit_status);
            }
            // Ottenere il codice di uscita del processo se disponibile
            if let Some(code) = exit_status.code() {
                println!("Codice di uscita del processo: {}", code);
            } else {
                if let Some(signal) = exit_status.signal() { // Utilizza ExitStatusExt
                    println!("Processo terminato dal segnale: {}", signal);
                }
                if exit_status.core_dumped() {
                    println!("Il processo ha generato un core dump");
                }
            }
        }
        Err(e) => { eprintln!("Errore durante l'attesa del processo: {}", e); }
    }
}

```



Test

- Proviamo il precedente programma con i seguenti test:
 - Un programma che termina con `exit(1)`
 - Un programma che termina con `exit(0)`
 - Un programma che termina con `abort()`
 - Un programma che termina con `panic!()`

```

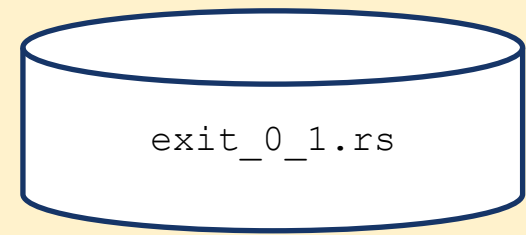
use std::process;
fn main() {
    println!("Esempio di programma in Rust che usa exit()");

    // Condizione per terminare il programma
    let some_condition = true;

    if some_condition {
        println!("Condizione soddisfatta, il programma terminerà con codice 1.");
        process::exit(1);
    }
    // Se il programma non è terminato prima, proseguirà con altre operazioni

    // Terminare il programma con codice 0 (successo)
    process::exit(0);
}

```



- Se il processo termina con **exit(1)**

- Il padre legge `ExitStatus(unix_wait_status(256))` => `exit_status.code()` ritorna il codice 1

Exited with exit code: exit status: 1
 Il processo è terminato con errore: `ExitStatus(unix_wait_status(256))`
 Codice di uscita del processo: 1

 Process finished with exit code 0

- Se il processo termina con **exit(0)**

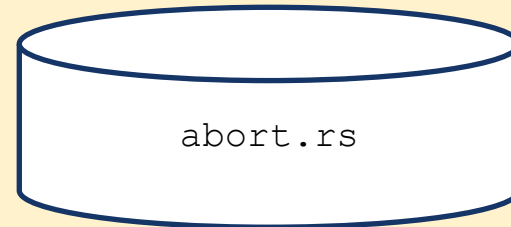
- Il padre legge `ExitStatus(unix_wait_status(0))` => `exit_status.code()` ritorna il codice 0

Exited with exit code: exit status: 0
 Il processo è terminato correttamente. `ExitStatus(unix_wait_status(0))`
 Codice di uscita del processo: 0

 Process finished with exit code 0

```
use std::process;
```

```
fn main() {  
    // Esegui alcune operazioni  
    println!("Esempio di programma in Rust che usa abort()");  
  
    // Condizione per terminare il programma con un abort  
    let some_condition = true;  
  
    if some_condition {  
        println!("Condizione soddisfatta, il programma terminerà con abort.");  
        process::abort();  
    }  
  
    // Se il programma non è terminato prima, proseguirà con altre operazioni  
    println!("Il programma continuerà a funzionare normalmente.");  
}
```



- Se il processo termina con **abort()**

- Il padre legge `ExitStatus(unix_wait_status(6)) => exit_status.signal()` ritorna il codice 6 corrispondente al signal SIGABRT.

Exited with exit code: signal: 6 (SIGABRT) (core dumped)
Il processo è terminato con errore: `ExitStatus(unix_wait_status(134))`
Processo terminato dal segnale: 6
Il processo ha generato un core dump

Process finished with exit code 0

```
fn main() {  
    // Esegui alcune operazioni  
    println!("Esempio di programma in Rust che usa panic!");  
  
    // Condizione per far scattare il panic  
    let some_condition = true;  
  
    if some_condition {  
        println!("Condizione soddisfatta, il programma terminerà con un panic.");  
        panic!("Errore: si è verificato un panic!");  
    }  
  
    // Se il programma non è terminato prima, proseguirà con altre operazioni  
    println!("Il programma continuerà a funzionare normalmente.");  
}
```



- Se il processo termina con **panic()**

- Il padre legge `ExitStatus(unix_wait_status(25856)) => exit_status.code()` ritorna il codice 101.

Exited with exit code: exit status: 101

Il processo è terminato con errore: `ExitStatus(unix_wait_status(25856))`

Codice di uscita del processo: 101

Process finished with exit code 0

Orfani e zombie

- Se un processo padre termina prima che l'ultimo dei suoi figli sia terminato, il processo figlio diventa **orfano**
 - Linux riassegna a tale figlio il PID 1 (init) come ID del processo padre
 - Questo può essere sfruttato da un processo per sapere se il proprio padre è ancora vivo o meno (ipotizzando che non sia stato lanciato direttamente da init)
- Se un processo figlio termina prima che il rispettivo padre abbia eseguito **wait*(...)** diventa uno zombie
 - La maggior parte delle sue risorse viene restituita al sistema operativo, tranne l'ID, lo stato di terminazione e i contatori relativi all'utilizzo delle risorse
 - Quando il padre effettua una **wait(...)** lo zombie viene rimosso

```
use std::process::{Command};
use std::thread;
use std::time::Duration;

fn main() {
    // Creazione di un processo figlio
    let mut child1 = Command::new("sleep")
        .arg("1")
        .spawn()
        .expect("failed to start child process");

    // Simulare un ritardo nel processo genitore
    println!("Waiting for the child process to finish...");
    thread::sleep(Duration::from_secs(2));

    let mut child2 = Command::new("ps")
        .arg("x")
        .spawn()
        .expect("failed to start child process");

    // Attesa del processo figlio per evitare che diventi zombie
    let status = child1.wait()
        .expect("failed to wait on child process");
    println!("Sleep process exited with status: {}", status);

    let status = child2.wait()
        .expect("failed to wait on child process");
    println!("ps process exited with status: {}", status);
}
```



Glossario

- Processo: istanza di un programma in esecuzione. Quando viene caricato in memoria, per l'esecuzione, un processo include diverse componenti: il codice del programma (text segment), i dati (data segment), lo stack, lo heap, lo stato del processore (il valore dei suoi registri), le risorse allocate (ad es. file aperti, connessioni di rete, dispositivi di I/O), lo stato del processo, l'ID del processo, informazioni per lo scheduler (priorità del processo e puntatori alle code di scheduling), informazioni per la gestione della memoria, informazioni di accounting,
- Isolamento: capacità di operare in modo indipendente senza interferire (e senza subire interferenze) da altri processi in esecuzione sullo stesso sistema.
- Protezione della memoria: ogni processo ha il proprio spazio di indirizzamento virtuale separato e isolato dagli altri.